Voice2Gov

Software Requirements Specification

Version 1.0

Citizen-Centric Civic Complaint Management Platform

Document Item	Specification
Project	Voice2Gov
Document	Software Requirements Specification (SRS)
Version	1.0
Frontend	React Native
Backend	Python FastAPI
Database	MongoDB
Primary Users	Citizens, Organizations, Administrators
Architecture	Three-Portals + REST API + MongoDB

Prepared for project development, review, academic submission, and stakeholder sharing.

Table of Contents

1. Introduction
2. System Objective
3. System Users and Portals
4. Technology Stack
5. High-Level Architecture
6. Core Complaint Workflow
7. Complaint Prioritization
8. Complaint Status Lifecycle
9. Functional Requirements
10. Authentication and Authorization
11. Backend REST API Specification
12. Organization APIs
13. Admin APIs
14. Notification APIs
15. React Native Frontend Requirements
16. Organization Portal Requirements
17. Admin Portal Requirements
18. MongoDB Database Design
19. Security Requirements
20. Performance and Reliability
21. Testing Requirements
22. Recommended Backend Project Structure
23. Recommended Mobile Project Structure
24. End-to-End Example
25. Complete API Catalogue
26. V1 Development Roadmap
27. Future Enhancements
28. Acceptance Criteria
29. Final System Definition

1. Introduction

Voice2Gov is a citizen-centric civic complaint management platform that creates a structured communication channel between citizens and government or civic organizations. Citizens can register complaints with photographs and location information, organizations can receive and manage those complaints, and administrators can manage the complete platform.

The platform is designed around transparency, community participation, department routing, priority management, complaint tracking, and measurable resolution.

2. System Objective

• Provide citizens with a simple mechanism to report local civic problems.

• Allow complaint evidence to be submitted using photographs.

• Route complaints to the appropriate organization and department.

• Expose eligible complaints through a community feed.

• Use community likes/upvotes as one input to complaint prioritization.

• Allow organizations to manage complaints through a defined lifecycle.

• Allow citizens to track complaint progress from submission to completion.

• Provide administrators with centralized control, organization management, and analytics.

• Maintain secure, auditable records of important system actions.

3. System Users and Portals

3.1 Admin Portal

• Create and manage organizations.

• Create and manage departments.

• Create organization login accounts.

• Manage users and access.

• Manage complaint categories.

• Monitor all complaints.

• View platform-wide analytics.

• Manage system configuration and audit information.

3.2 Organization Portal

• Each organization has an independent login.

• View complaints routed to the organization.

• Filter, search, and prioritize complaints.

• Review complaint photographs and location.

• Assign complaints to departments or staff.

• Update complaint status.

• Add progress updates and evidence.

• Mark resolved complaints as completed.

• View organization analytics.

3.3 Citizen Portal

• Register and authenticate.

• Register civic complaints.

• Capture or upload photographs.

• Provide complaint description and location.

• View the community feed.

• Like/upvote complaints.

• View and track personal complaints.

• Receive notifications.

• View complaint status timeline and history.

4. Technology Stack

Layer	Technology	Purpose
Mobile Frontend	React Native	Citizen mobile application
Backend	Python + FastAPI	REST API, business logic, validation, authentication
Database	MongoDB	Users, organizations, departments, complaints, likes, notifications, audit data
API Format	REST + JSON	Client-server communication
Authentication	JWT + Role-Based Access Control	Secure access and authorization
Image Upload	Multipart upload + secure storage / GridFS	Complaint and progress photographs
Notifications	Push notification service	Complaint and system notifications
API Documentation	OpenAPI / Swagger	Backend API documentation and testing

5. High-Level Architecture

```
Citizen React Native App | | REST / JSON / Multipart v Python FastAPI Backend | +---- Authentication / RBAC +---- Complaint Service +---- Organization Service +---- Notification Service +---- Priority Service +---- Admin Service | v MongoDB | +---- users +---- organizations +---- departments +---- complaints +---- complaint_likes +---- complaint_updates +---- notifications +---- categories +---- audit_logs
```

The organization and admin portals may be web applications consuming the same FastAPI backend. The citizen-facing portal is implemented in React Native.

6. Core Complaint Workflow

```
User Login -> Home -> Register Complaint -> Select Category -> Capture / Upload Photo -> Enter Description -> Confirm Location -> Submit -> Backend Validation -> Organization / Department Routing ->
```

```
Complaint Stored in MongoDB -> Complaint Appears in Community Feed -> Community Likes / Upvotes ->
Priority Score Updated -> Organization Receives Complaint -> Ongoing -> Processing -> Completed ->
Citizen Receives Notifications
```

The exact organization/department routing may initially be category-based and later be enhanced using location rules or AI-assisted classification.

7. Complaint Prioritization

Community engagement is a core feature of Voice2Gov. Likes/upvotes increase the visibility and priority of a complaint. Likes should not be the only priority factor; the recommended design combines community engagement, severity, age, and organization-configured priority.

```
Priority Score = Community Engagement Score + Severity Score + Complaint Age Score + Organization
Priority Score
```

The formula and weights should be configurable so that the system can evolve without changing the mobile application.

8. Complaint Status Lifecycle

Status	Meaning	Typical Actor
SUBMITTED	Complaint has been successfully registered.	Citizen/System
ONGOING	Organization has acknowledged the complaint and work has started.	Organization
PROCESSING	Complaint is actively being investigated or resolved.	Organization
COMPLETED	Work has been completed and the complaint is resolved.	Organization
REJECTED	Complaint was rejected with a reason.	Organization/Admin
DUPLICATE	Complaint duplicates an existing complaint.	Organization/Admin
CLOSED	Complaint lifecycle is administratively closed.	Organization/Admin

Primary citizen-facing flow: SUBMITTED -> ONGOING -> PROCESSING -> COMPLETED. Backend validation must prevent invalid transitions such as COMPLETED -> SUBMITTED.

9. Functional Requirements

9.1 Citizen Complaint Registration

• User must be authenticated.

• User can select a complaint category.

• User can enter a title and description.

• User can capture a photograph using the camera.

• User can select an image from the gallery.

• User can confirm or provide location.

• System validates the request before creating the complaint.

• System routes the complaint to the appropriate organization/department.

• User receives a complaint identifier after successful creation.

9.2 Community Feed

• Display public/eligible complaints.

• Support popular, recent, nearby, and priority sorting.

• Display image, title, description, location, status, and engagement count.

• Allow a user to like/unlike a complaint.

• Prevent duplicate likes by the same user.

9.3 Complaint Tracking

• Display current status.

• Display organization and department.

• Display timeline of status changes.

• Display progress updates.

• Display completion information.

• Send notifications when important state changes occur.

10. Authentication and Authorization

JWT-based authentication is recommended. Access tokens should be short-lived and refresh tokens should be managed securely.

Role	Access
admin	Full platform administration
organization_admin	Own organization, departments, complaints, analytics, staff
organization_staff	Assigned/authorized organization complaints and updates
citizen	Own profile/complaints, public feed, likes, notifications

• Passwords must never be stored in plaintext.

• Use Argon2 or bcrypt for password hashing.

• Enforce role-based authorization at the API layer.

• Validate organization ownership on organization APIs.

• Protect administrative endpoints with explicit admin authorization.

11. Backend REST API Specification

11.1 Authentication

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Citizen registration
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/logout	Logout/session invalidation

Method	Endpoint	Purpose
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/auth/forgot-password	Start password recovery
POST	/api/v1/auth/reset-password	Reset password
POST	/api/v1/auth/verify-otp	Verify OTP
GET	/api/v1/auth/me	Current authenticated user

11.2 User APIs

Method	Endpoint	Purpose
GET	/api/v1/users/me	Get profile
PUT	/api/v1/users/me	Update profile
POST	/api/v1/users/me/avatar	Upload profile image
DELETE	/api/v1/users/me	Delete account

11.3 Complaint APIs

Method	Endpoint	Purpose
POST	/api/v1/complaints	Create complaint
GET	/api/v1/complaints/{id}	Get complaint details
GET	/api/v1/complaints/my	Get current user's complaints
PUT	/api/v1/complaints/{id}	Update permitted complaint fields
DELETE	/api/v1/complaints/{id}	Delete complaint when allowed
POST	/api/v1/complaints/{id}/images	Upload complaint image
GET	/api/v1/complaints/{id}/priority	Get priority information

11.4 Like / Upvote APIs

Method	Endpoint	Purpose
POST	/api/v1/complaints/{id}/like	Like/upvote
DELETE	/api/v1/complaints/{id}/like	Remove like
GET	/api/v1/complaints/{id}/like-status	Current user's like status
GET	/api/v1/complaints/{id}/likes	Like information/count

11.5 Complaint Update APIs

Method	Endpoint	Purpose
POST	/api/v1/complaints/{id}/updates	Add progress update
GET	/api/v1/complaints/{id}/updates	Get complaint history/updates

Method	Endpoint	Purpose
PATCH	/api/v1/complaints/{id}/status	Change complaint status
PATCH	/api/v1/complaints/{id}/complete	Complete complaint
PATCH	/api/v1/complaints/{id}/assign	Assign complaint

11.6 Community Feed

Method	Endpoint	Purpose
GET	/api/v1/feed	Community feed with pagination, sorting and filtering

12. Organization APIs

Method	Endpoint	Purpose
GET	/api/v1/organizations	List organizations
GET	/api/v1/organizations/{id}	Get organization
POST	/api/v1/organizations	Create organization; admin only
PUT	/api/v1/organizations/{id}	Update organization
PATCH	/api/v1/organizations/{id}/status	Enable/disable organization
GET	/api/v1/organizations/me/complaints	Organization complaint queue
GET	/api/v1/organizations/me/complaints/priority	Priority queue

Organization complaint APIs must enforce tenant isolation: an organization may access only records belonging to its organization unless the caller has global administrative permission.

13. Admin APIs

Method	Endpoint	Purpose
GET	/api/v1/admin/dashboard	Platform-wide dashboard
GET	/api/v1/admin/users	List/search users
GET	/api/v1/admin/users/{id}	Get user
PATCH	/api/v1/admin/users/{id}/suspend	Suspend user
PATCH	/api/v1/admin/users/{id}/activate	Activate user
POST	/api/v1/admin/categories	Create category
PUT	/api/v1/admin/categories/{id}	Update category
DELETE	/api/v1/admin/categories/{id}	Delete category

14. Notification APIs

Method	Endpoint	Purpose
GET	/api/v1/notifications	Get notifications
GET	/api/v1/notifications/unread-count	Unread count
PATCH	/api/v1/notifications/{id}/read	Mark notification read
PATCH	/api/v1/notifications/read-all	Mark all read
POST	/api/v1/notifications/devices	Register push device

• Notify citizen when complaint is submitted, assigned, status-changed, or completed.

• Notify organization when a new/high-priority complaint is received.

• Use push notifications plus an in-app notification center.

15. React Native Frontend Requirements

15.1 Main Navigation

```
Splash -> Login / Register -> Home |-> Community Feed |-> Register Complaint |-> My Complaints |-> Notifications |-> Profile
```

15.2 Home Page

• Greeting and profile context.

• Quick Register Complaint action.

• Complaint summary statistics.

• Recent complaint updates.

• Community feed preview.

• Notification indicator.

15.3 Register Complaint

• Category selection.

• Title and description.

• Camera integration.

• Gallery selection.

• Image preview/removal.

• Location confirmation.

• Submission validation.

• Upload progress.

• Success/error state.

15.4 Community Feed

• Complaint cards.

• Image preview.

• Like count and like action.

• Status.

• Location.

• Popular/recent/nearby/priority filters.

• Pagination/infinite scrolling.

15.5 My Complaints

• All, Ongoing, Processing, Completed tabs.

• Complaint card list.

• Search/filter.

• Complaint detail navigation.

15.6 Complaint Details

• Image.

• Title and description.

• Location.

• Organization and department.

• Likes.

• Priority.

• Current status.

• Timeline.

• Progress updates.

• Completion evidence.

15.7 Notifications

• Chronological notifications.

• Unread indicator.

• Mark read.

• Open related complaint from notification.

15.8 Profile

• Profile information.

• Profile image.

• Complaint statistics.

• Settings.

• Privacy.

• Logout.

16. Organization Portal Requirements

• Organization login.

- Dashboard KPIs.
- Complaint queue.
- Priority queue.
- Complaint detail view.
- Image and location review.
- Department assignment.
- Staff assignment.
- Status management.
- Progress updates.
- Completion evidence.
- Filters by category/status/date/department.
- Organization analytics.
- Notification center.

16.1 Organization Dashboard KPIs

- Total complaints.
- Submitted/pending.
- Ongoing.
- Processing.
- Completed.
- High priority.
- Average resolution time.
- Resolution rate.

17. Admin Portal Requirements

- Admin login.
- Organization creation/editing/disablement.
- Department management.
- Category management.
- User management.
- Global complaint monitoring.
- Platform analytics.
- Audit log review.
- System configuration.

17.1 Suggested Admin Analytics

- Total citizens.
- Total organizations.

• Total departments.

• Total complaints.

• Complaint volume over time.

• Complaints by category.

• Complaints by organization.

• Complaints by area.

• Resolution rate.

• Average resolution time.

18. MongoDB Database Design

Recommended collections:

```
users organizations departments complaints complaint_images complaint_likes complaint_updates
notifications categories device_tokens audit_logs admin_settings
```

18.1 Users

```
{ _id, name, email, phone, password_hash, role, status, profile_image, created_at, updated_at }
```

18.2 Organizations

```
{ _id, name, code, description, contact_email, status, created_at, updated_at }
```

18.3 Departments

```
{ _id, organization_id, name, code, status, created_at }
```

18.4 Complaints

```
{ _id, user_id, organization_id, department_id, category_id, title, description, images, location: {
type: "Point", coordinates: [longitude, latitude], address }, status, likes_count, priority_score,
assigned_to, created_at, updated_at, completed_at }
```

18.5 Complaint Likes

```
{ _id, complaint_id, user_id, created_at }
```

Create a unique compound index on complaint_id + user_id to prevent duplicate likes.

18.6 Complaint Updates

```
{ _id, complaint_id, organization_id, updated_by, previous_status, new_status, message, images,
created_at }
```

18.7 Notifications

```
{ _id, user_id, type, title, message, complaint_id, is_read, created_at }
```

18.8 Audit Logs

```
{ _id, actor_id, actor_role, action, entity_type, entity_id, metadata, created_at }
```

18.9 MongoDB Indexing

• users.email

• users.phone

• complaints.user_id

• complaints.organization_id

• complaints.department_id

• complaints.status

• complaints.category_id

• complaints.created_at

• complaints.priority_score

• complaints.location as a 2dsphere index

• complaint_likes complaint_id + user_id

• notifications user_id + is_read

• complaint_updates complaint_id + created_at

19. Security Requirements

• JWT access/refresh token architecture.

• Argon2 or bcrypt password hashing.

• Role-based authorization.

• Organization-level tenant isolation.

• Input validation through Pydantic schemas.

• File MIME type and size validation.

• Filename sanitization.

• Rate limiting for login, OTP, uploads, likes, and password reset.

• Secure transport using HTTPS.

• No sensitive citizen data in public feed.

• Audit logs for important administrative and organization actions.

Image files should normally be stored in secure object storage or GridFS while MongoDB stores the reference/URL and metadata.

20. Performance and Reliability

• Target normal API response time below 500 ms where practical.

• Target normal feed loading below 2 seconds under normal network conditions.

• Use pagination for all large lists.

• Use MongoDB indexes for common queries.

• Use asynchronous FastAPI operations where appropriate.

• Process heavy image/notification tasks asynchronously.

• Implement retries for transient notification/storage failures.

• Provide meaningful error responses and frontend retry states.

20.1 Standard API Response

```
{ "success": true, "message": "Complaint created successfully", "data": {} }
```

20.2 Standard API Error

```
{ "success": false, "message": "Complaint not found", "error_code": "COMPLAINT_NOT_FOUND", "errors": []
}
```

Recommended status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, and 500.

21. Testing Requirements

21.1 Unit Testing

• Authentication and authorization.

• Complaint validation.

• Priority calculation.

• Like/unlike logic.

• Status transition rules.

• Notification generation.

• Database repository/service logic.

21.2 API Testing

• Valid requests.

• Invalid requests.

• Unauthorized requests.

• Forbidden requests.

• Missing resources.

• Duplicate requests.

• Database failures.

• File upload failures.

21.3 Security Testing

• JWT validation.

• Role bypass attempts.

• Cross-organization access attempts.

• File upload security.

• Injection attacks.

• Rate limiting.

• Token expiry.

• Password security.

FastAPI OpenAPI/Swagger documentation should be used during API development and testing.

22. Recommended Backend Project Structure

```
backend/ app/ main.py api/v1/ auth.py users.py complaints.py organizations.py departments.py
categories.py feed.py notifications.py admin.py models/ schemas/ services/ repositories/ middleware/
security/ utils/ database/ tests/ requirements.txt .env
```

23. Recommended Mobile Project Structure

```
mobile/ src/ screens/ auth/ home/ complaints/ feed/ notifications/ profile/ components/ navigation/
services/ api/ auth/ notifications/ store/ hooks/ utils/ constants/ assets/ App.tsx package.json
```

24. End-to-End Example

A citizen notices sewage overflow. The citizen logs in, selects Register Complaint, selects Sewage, captures a photograph, enters a description, confirms the location, and submits. FastAPI validates the request and routes it to the municipality/sewage department. The complaint is stored in MongoDB and appears in the citizen's My Complaints, the community feed, and the organization's dashboard.

Other citizens can like the complaint. As engagement increases, the priority score may increase. The organization receives a notification, acknowledges the complaint, changes it to ONGOING, later changes it to PROCESSING, adds progress information, and finally marks it COMPLETED. The citizen receives notifications at the important status transitions.

25. Complete API Catalogue

Area	Methods and endpoints
Authentication	POST /api/v1/auth/register; POST /api/v1/auth/login; POST /api/v1/auth/logout; POST /api/v1/auth/refresh; POST /api/v1/auth/forgot-password; POST /api/v1/auth/reset-password; POST /api/v1/auth/verify-otp; GET /api/v1/auth/me
Users	GET /api/v1/users/me; PUT /api/v1/users/me; POST /api/v1/users/me/avatar; DELETE /api/v1/users/me
Complaints	POST /api/v1/complaints; GET /api/v1/complaints/{id}; GET /api/v1/complaints/my; PUT /api/v1/complaints/{id}; DELETE /api/v1/complaints/{id}; POST /api/v1/complaints/{id}/images; GET /api/v1/complaints/{id}/priority
Likes	POST /api/v1/complaints/{id}/like; DELETE /api/v1/complaints/{id}/like; GET /api/v1/complaints/{id}/like-status; GET /api/v1/complaints/{id}/likes
Updates	POST /api/v1/complaints/{id}/updates; GET /api/v1/complaints/{id}/updates; PATCH /api/v1/complaints/{id}/status; PATCH /api/v1/complaints/{id}/complete; PATCH /api/v1/complaints/{id}/assign
Feed	GET /api/v1/feed
Organizations	GET /api/v1/organizations; GET /api/v1/organizations/{id}; POST /api/v1/organizations; PUT /api/v1/organizations/{id}; PATCH /api/v1/organizations/{id}/status; GET /api/v1/organizations/me/complaints; GET /api/v1/organizations/me/complaints/priority
Departments	GET /api/v1/departments; GET /api/v1/departments/{id}; POST /api/v1/departments; PUT /api/v1/departments/{id}; DELETE /api/v1/departments/{id}
Categories	GET /api/v1/categories; POST /api/v1/admin/categories; PUT /api/v1/admin/categories/{id}; DELETE /api/v1/admin/categories/{id}
Notifications	GET /api/v1/notifications; GET /api/v1/notifications/unread-count; PATCH /api/v1/notifications/{id}/read; PATCH /api/v1/notifications/read-all; POST /api/v1/notifications/devices
Location	GET /api/v1/location/reverse

Area	Methods and endpoints
Search	GET /api/v1/search
Admin	GET /api/v1/admin/dashboard; GET /api/v1/admin/users; GET /api/v1/admin/users/{id}; PATCH /api/v1/admin/users/{id}/suspend; PATCH /api/v1/admin/users/{id}/activate

26. V1 Development Roadmap

Phase	Main Deliverables
1. Foundation	FastAPI, MongoDB, authentication, JWT, RBAC, schemas, repositories
2. Citizen	React Native authentication, home, complaint registration, camera/image, location, my complaints
3. Organization	Organization login, dashboard, complaint queue, assignment, status, progress, completion
4. Community	Community feed, likes, priority scoring, search and filters
5. Notifications	Push notifications and in-app notification center
6. Admin	Admin dashboard, organizations, departments, users, categories, analytics, audit logs
7. Advanced	AI classification, duplicate detection, automatic routing, heat maps, multilingual/offline features

27. Future Enhancements

• AI-based complaint classification.

• Automatic severity detection.

• AI-assisted image analysis.

• Duplicate complaint detection.

• Voice-based complaint registration.

• Tamil/English multilingual support.

• Live complaint map and heat maps.

• Government SLA tracking and escalation.

• Officer performance analytics.

• Geo-fencing.

• SMS/WhatsApp integration.

• Citizen-organization chat.

• Public transparency dashboard.

28. Acceptance Criteria

28.1 Citizen

• Can register/login.

• Can submit a complaint with image and location.

• Can see complaint in My Complaints.

• Can view eligible complaints in the community feed.

• Can like/unlike complaints.

• Can track status and timeline.

• Receives relevant notifications.

28.2 Organization

• Can log in securely.

• Can see only authorized complaints.

• Can sort by priority.

• Can inspect evidence.

• Can assign complaints.

• Can change status through valid transitions.

• Can add progress updates.

• Can complete complaints.

• Can view organization analytics.

28.3 Admin

• Can create and manage organizations.

• Can create and manage departments.

• Can manage categories.

• Can manage users.

• Can monitor complaints.

• Can view global analytics.

• Can access audit information.

29. Final System Definition

Voice2Gov is a three-portal civic governance platform. The React Native citizen application provides complaint registration, image capture, location, community participation, complaint tracking, and notifications. The organization portal provides complaint intake, prioritization, assignment, status management, progress tracking, completion, and analytics. The admin portal provides organization, department, user, category, configuration, and platform-level management.

The central workflow is: Citizen -> Complaint -> Image + Location -> Department Routing -> Community Feed -> Community Likes -> Priority -> Organization -> Assignment -> Ongoing -> Processing -> Completed -> Citizen Notification.

The core technology stack is React Native for the citizen frontend, Python FastAPI for the backend, and MongoDB for persistence, with REST/JSON APIs, JWT authentication, role-based access control, secure image storage, and push notifications.

The core product principle is: A citizen reports a problem once, the right organization receives it, the community helps prioritize it, and the citizen can transparently follow its resolution from submission to completion.